

Thyra

Quality Gates for an Energy Forecasting Platform

Bledar (Blake) Zenuni | October 5, 2026

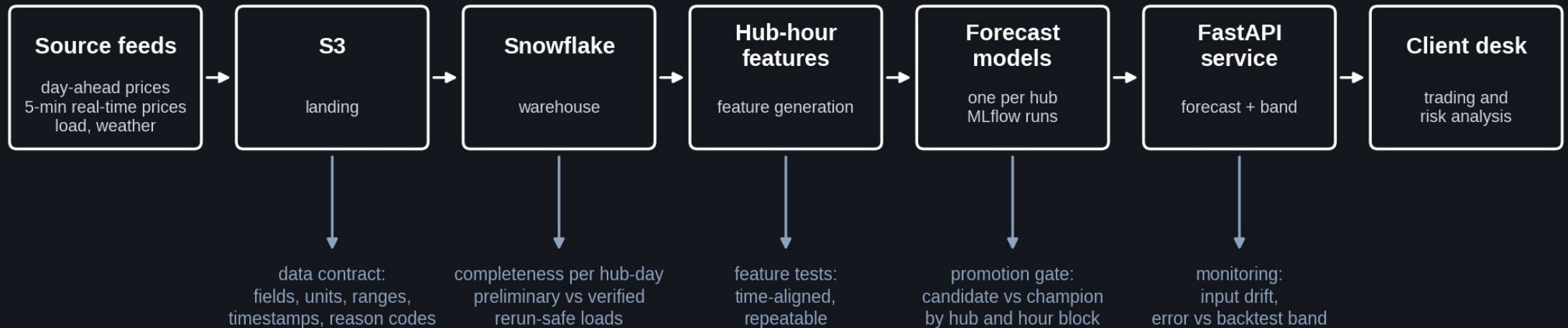
The Project and My Role

- The platform forecast day-ahead (DA) and real-time (RT) energy prices for a client of my previous employer.
- The forecast informed the client's trading and risk decisions, but it did not make them.
- The hard part was proving the data, the pipeline and each released model were fit for use.
- I was the technical program manager (TPM) and the team's only QA engineer: I designed and built the validation, release-gate and service controls.

Goals and Objectives

- Produce an hourly forecast per market hub with a **stated prediction interval**.
- Block bad or incomplete data before feature generation and scoring.
- Promote a retrained model only when it beat the deployed model on a fair test.
- Detect data and forecast failures before users reported them.

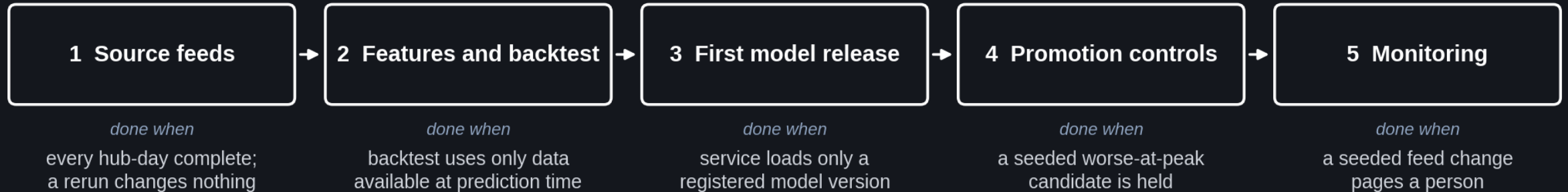
The Platform



A quality control at each handoff. MLflow holds every run and approval; GitLab CI runs the tests and the gate.

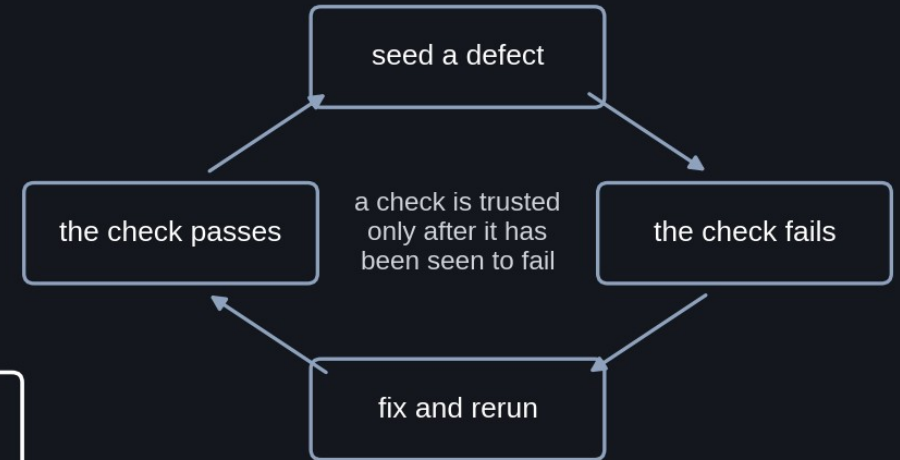
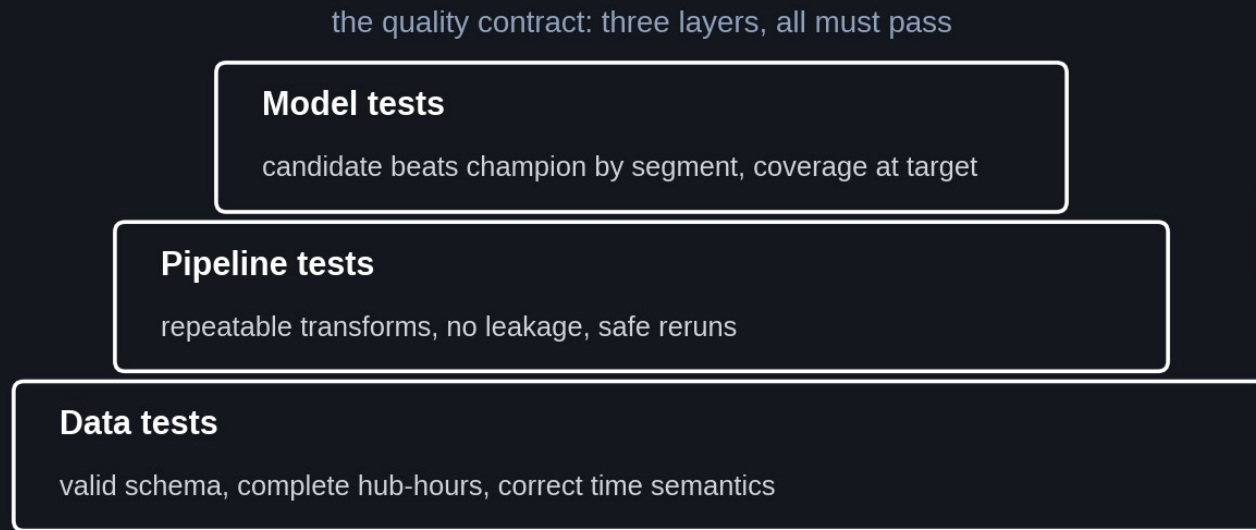
- The grid operator published DA prices, five-minute RT prices, load and weather: gigabytes per day across hubs.
- Data moved from S3 to Snowflake, then through feature generation and per-hub models to a FastAPI service.
- I placed a control at each handoff: data validation, MLflow run records, Gitlab CI release gates.

Planning and Execution



- I planned five stages: source feeds, features and backtesting, first release, promotion controls and monitoring.
- Each stage had a testable exit condition, and a convincing demo was not one of them.
- Every change entered through a merge request (MR) that I reviewed for quality impact.
- Before trusting a new control, I seeded the failure it was built to catch.

Test Strategy



- A model advanced only when all three layers passed: a strong forecast did not compensate for untrusted data or an unrepeatable pipeline.
- Fast data and pipeline tests **ran in CI on every MR**; the **full model-promotion backtest** ran nightly on demand.
- I trusted a check only after I had seen it fail on a seeded defect.

Data Validation



- Each record passed a typed contract, a completeness count and a reconciliation against the verified prices.
- Refused records were quarantined with a reason code, and a repeated load changed nothing.

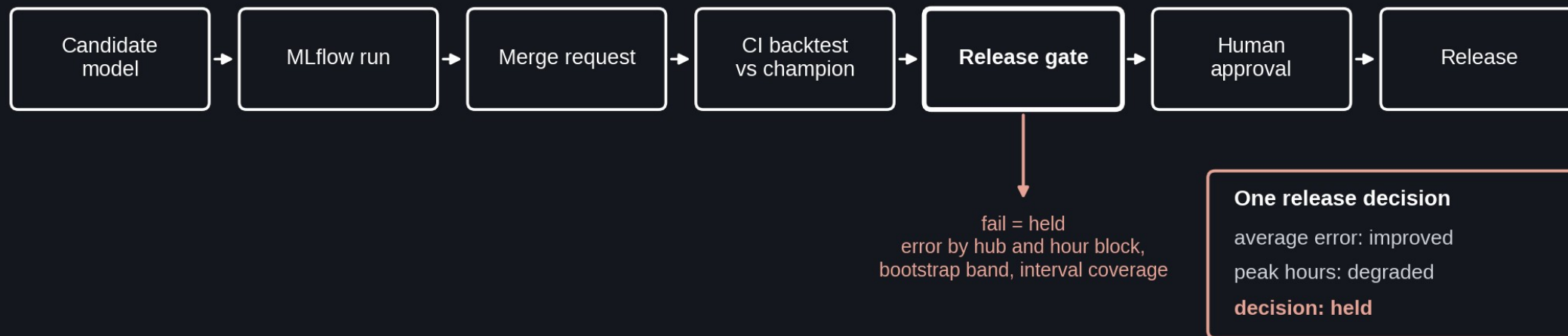
Challenge 1: Time and Data Availability



fix: the backtest reads the same information set production had. reported accuracy then equals deployable accuracy.

- Two kinds of data were correct but unusable.
- UTC in the warehouse and local time in one notebook shifted forecasts by one hour on the daylight-saving day.
- I standardized on UTC and added tests for the 23-hour day, the 25-hour day and the UTC round trip.
- Backtests had read verified prices that production could not see yet, so I made them read the preliminary prices available at prediction time.

Challenge 2: Model Promotion



- At first the service could load a retrained model file directly.
- I replaced that with a controlled path: MLflow run, MR, CI backtest against the champion, release gate and a person's approval.
- The gate compared error by hub and hour block with a **paired-bootstrap interval for the MAE difference**, then checked prediction-interval coverage against the target rate.
- One candidate improved average error but degraded peak hours; **the gate blocked its promotion.**

Thyra: The Public Copy

- Thyra is a **public implementation** of the same quality pattern.
- It pulls DA and RT prices from public ISO data, saves a fixed snapshot, validates the data and runs a promotion gate.
- It contains no client code, data or names.

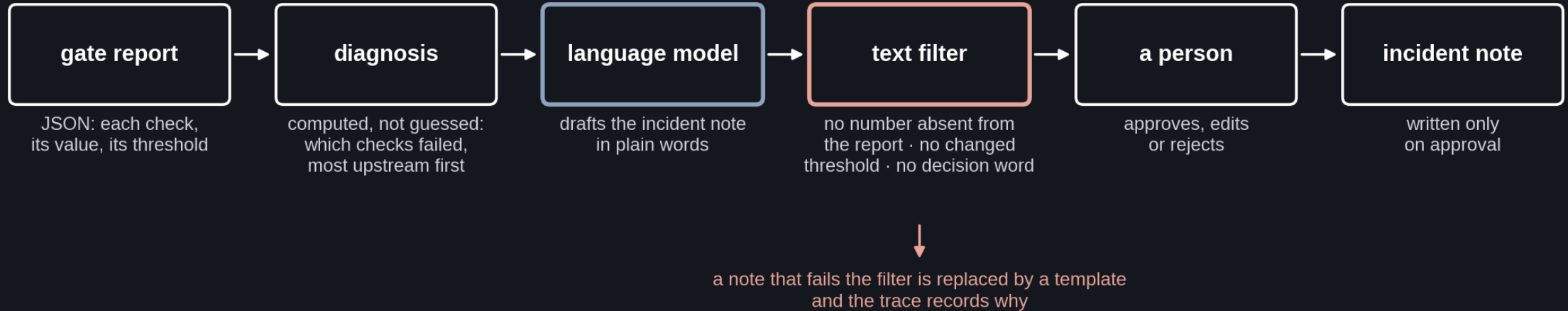
Demo, Part One: QA

check	value	threshold	result
quarantine_rate	0.0000	<= 0.005	pass
hour_count_days_wrong	1	== 0	FAIL
duplicate_conflicts	1	== 0	FAIL
reconcile_share_off	0.0090	<= 0.02	pass
reconcile_mean_abs	0.1022	<= 0.5	pass
overall_improvement	2.1241	< 0.0	FAIL
peak_no_regression	2.0799	<= 0.0	FAIL
hub_no_regression	3.1414	<= 0.0	FAIL
band_coverage	0.8172	within [0.85, 0.95]	FAIL
psi_max	0.1236	<= 0.25	pass
4/10 passed -> exit 2 (release held)			

- First I run the timestamp and contract tests.
- Next the gate runs on a clean snapshot and passes with exit code 0.
- Then I seed three defects, and the same gate fails with exit code 2 and names the control that failed.

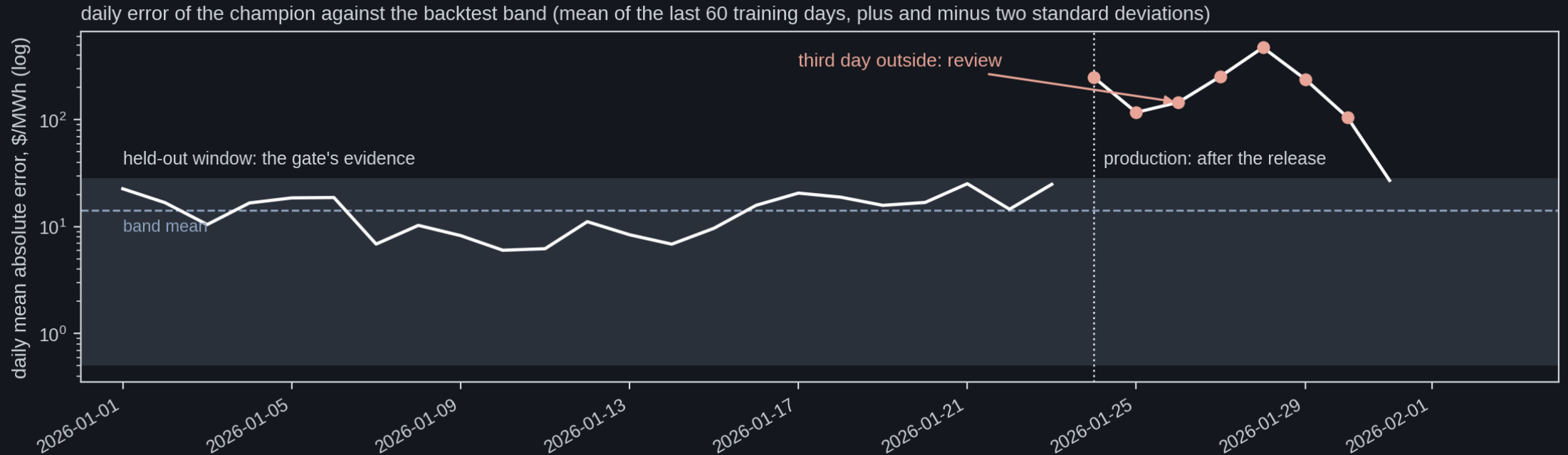
Demo, Part Two: AI

the model restates the evidence: it cannot alter the evidence, the thresholds or the release decision



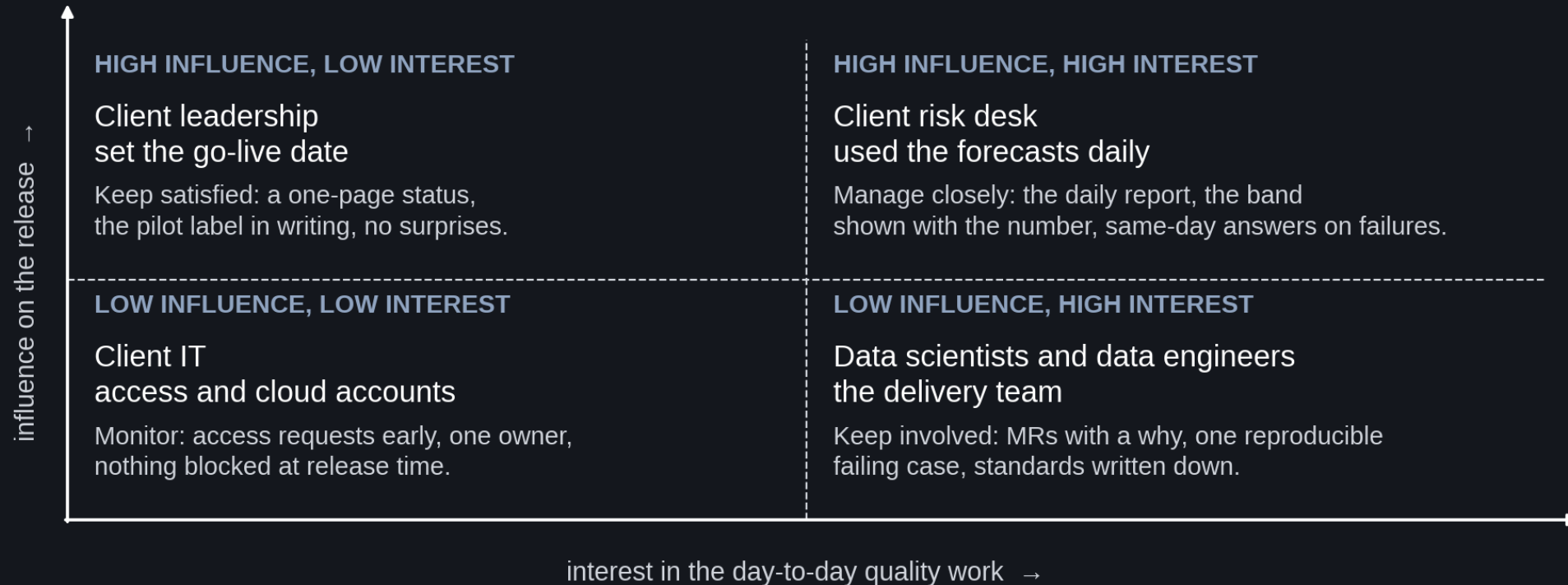
- The gate report goes to a language model, which drafts the incident note in plain words.
- A text filter and a person review the draft: the model cannot alter the structured evidence, the thresholds, or the release decision.

Monitoring



- Monitoring tested whether production still resembled the conditions the model was approved under.
- I tracked input drift per feature with the **Population Stability Index (PSI)** and compared daily error with the backtest range.
- Drift was read in the daily report, and the alarm was three consecutive days of error outside the range.
- In a winter cold snap the error left the range, monitoring flagged it within a day and we retrained with wider intervals.

Team and stakeholders



- The delivery team was six people: two data scientists, two data engineers, one analyst and me as the quality owner.
- A go-live date was set before the final backtest finished, so I kept the gate on and documented the release as a limited pilot.

My role and Limits

Designed	Where validation ran, what could block a release and the standards for reviewing changes.
Built	The validation module, the CI promotion gate, the FastAPI service and the Bedrock incident-note workflow.
Reviewed	Model behavior and Snowflake transformations. The data scientists and data engineers owned the implementation.
Limits	A small hub set, one grid operator, one model family and a gate that took about a day to run.

Close

- The project changed the question from "does the forecast look reasonable?" to "what evidence lets us trust this release?"
- My contribution was to make that evidence automated, reproducible and visible to the people who decided.
- The pattern transfers: validate inputs, test transformations, evaluate candidates under **production-realistic conditions, block unsafe releases and monitor production after release.**